

Playwright-Notes-7

- **Codeless Automation with Playwright Codegen-**
- Playwright comes with the ability to generate tests for you as you perform actions in browser and is a great way to get started quickly without testing
- Playwright will look at your page and figure out the best locator, prioritizing **role, text and test id locators-** if the generator finds multiple elements matching the locator, it will improve the locator to make it resilient that uniquely identifies the target element
- Update Playwright with the command: **npm install -D @playwright/test@latest**
- Then install the new browsers: **npx playwright install**
- **Recording the test using Playwright VSCode Extension-** Playwright will record your actions and generate the test code directly in VSCode. Click on the **Testing icon** below the **Extensions icon** in VSCode. Below the **Playwrights section** click on **Record New** option. Clicking this option will open a new browser which allows you to **start/stop the recording, pick a locator of an element, assert the visibility of an element, assert whether an element contains a specific text or not, assert whether an element contains a specific value or not. All actions performed in the browser will be recorded in the form of test scripts in a file called test-1.spec.ts that will automatically created.**
- You can also record the test by using the command: **npx playwright codegen** in your terminal (**powershell**). This will open a new browser for you to start recording your tests
- **We can change the language (typescript, java, javascript, python) in which the test scripts are getting generated using the Playwright Inspector window**
- Using the **Pick Locator** option under the **Playwrights section** when clicking on the **Testing icon** allows us to obtain the location of an element-it opens a new browser, allows us to click on the element to get the location and then we can copy the location of the element from the **Playwrights tab** near the **terminal** at the bottom
- **Debugging in Playwright in VSCode-** debugging a test script is the process of finding and fixing errors in a script-to debug all tests run the Playwright test command followed by the **--debug** flag: **npx playwright test --debug**
- We can also debug tests using the **VSCode extension under the Testing icon where we run our tests**, and this allows us to see error messages, create breakpoints and live debug our tests

- **Playwright Trace Viewer in VSCode-** Trace viewer lets you view traces of your tests meaning you can go back and forward through each action of your test and visually see what was happening during each action
- Ways to access a generated trace-**Click on the checkbox of the Show Trace option under the Playwright section in the Testing tab-** this will allow you to view the trace of your test in the Trace Viewer (or) in your terminal (powershell) type the command `npx show-trace`. This will allow you to select a file for viewing the trace-this is a zip file which can be selected from the test-results folder in your current project/directory (or) after running your test open the index.html file under the playwright-report folder to view the HTML report of your test (or) open the trace.playwright.dev website and this will allow you to select your zip file (in the test-results) folder for which you want to view your trace (or) we can also specify the settings in the `playwright.config.ts` for Trace
- We can view the following in a trace (components in a trace):

- **'on-first-retry'** - Record a trace only when retrying a test for the first time.
- **'on-all-retries'** - Record traces for all test retries.
- **'off'** - Do not record a trace.
- **'on'** - Record a trace for each test. (not recommended as it's performance heavy)
- **'retain-on-failure'** - Record a trace for each test, but remove it from successful test runs.

Different Sections in Trace -

Actions - In the Actions tab you can see what locator was used for every action.

Timeline - each trace records a screencast and renders it as a film strip.

Snapshots - Playwright captures a set of complete DOM snapshots for each action.

Source - As you hover over each action of your test the line of code for that action is highlighted in the source panel.

Call - The call tab shows you information about the action such as the time it took, what locator was used, if in strict mode and what key was used.

Log - See a full log of your test to better understand what Playwright is doing behind the scenes such as scrolling into view, waiting for element to be visible, enabled and stable and performing actions such as click, fill, press etc.

Errors - If your test fails you will see the error messages for each test in the Errors tab.

Console - See console logs from the browser as well as from your test.

Metadata - It will show you more information on your test such as the Browser, viewport size, test duration and more.

Attachments - The "Attachments" tab allows you to explore attachments. If you're doing visual regression testing, you'll be able to compare screenshots

- **UI mode in Playwright-** UI mode lets you explore, run and debug tests with a time travel experience with watch mode- we can see full trace of your tests and hover back and forward over each action to see what was happening during each step and pop out the DOM snapshot to a separate window for a better debugging experience-to activate UI mode use the command `npx playwright test --ui`

Different Sections in UI Mode-

Watch Mode - Next to the name of each test in the sidebar you will find an eye icon. Clicking on the icon will activate watch mode which will re-run the test when you make changes to it.

Actions - In the Actions tab you can see what locator was used for every action.

Timeline - each trace records a screencast and renders it as a film strip.

Snapshots - Playwright captures a set of complete DOM snapshots for each action.

Source - As you hover over each action of your test the line of code for that action is highlighted in the source panel.

Call - The call tab shows you information about the action such as the time it took, what locator was used, if in strict mode and what key was used.

Log - To see a full log of your test to better understand what Playwright is doing behind the scenes such as scrolling into view, waiting for element to be visible, enabled and stable and performing actions such as click, fill, press etc.

Errors - If your test fails you will see the error messages for each test in the Errors tab.

Console - See console logs from the browser as well as from your test.

Network - The Network tab shows you all the network requests that were made during your test.

Metadata - It will show you more information on your test such as the Browser, viewport size, test duration and more.

Attachments - The "Attachments" tab allows you to explore attachments. If you're doing visual regression testing, you'll be able to compare screenshots

Save and Reuse Authentication state | Playwright Tutorials

Playwright executes tests in isolated environments called browser contexts. By reusing authentication state tests can load existing authenticated state. This eliminates the need to authenticate in every test and speeds up test execution.

- Here we will authenticate once in the setup project, save the authentication state, and then reuse it to bootstrap each test already authenticated.
- We will create a new setup project in the config file and declare it as a dependency for all your testing projects. This project will always run and authenticate before all the tests. All testing projects should use the authenticated state as storageState.
- Reusing authenticated state covers cookies and local storage based authentication.

Cookies - An HTTP cookie (web cookie, browser cookie) is a small piece of data that a server sends to a user's web browser. The browser may store the cookie and send it back to the same server with later requests. Typically, an HTTP cookie is used to tell if two requests come from the same browser—keeping a user logged in,

Below are 3 different ways to Save / Preserve and Reuse Authentication state.

1- By using Codegen command - Run codegen with `--save-storage` to save cookies and localStorage at the end of the session. This is useful to separately record an authentication step and reuse it later when recording more tests.

2 - By using globalSetup - We can use the globalSetup option in the configuration file to set something up once before running all tests. Here it will authenticate once and reuses the authentication state in tests.

3 - By using Dependencies - Project dependencies are a list of projects that need to run before the tests in another project run. With project dependencies, you define a project that runs before all other projects. This is the recommended way to configure global setup as with Project dependencies your HTML report will show the global setup, trace viewer will record a trace of the setup and fixtures can be used.

Avoid authentication in some tests - We can reset storage state in a test file to avoid authentication that was set up for the whole project.

```
test.use({ storageState: { cookies: [], origins: [] } });
```

Avoid authentication is specific test only -

```
await context.clearCookies();
```

Note - Regardless of the authentication strategy you choose, you are likely to store authenticated browser state on the file system. It is recommended to create playwright/.auth directory and add it to your .gitignore.

Codegen commands to use step-by-step:

1. **npx playwright codegen --save-storage=<your file name in which you want to save the cookies (this is optional)>**
2. **npx playwright codegen --load-storage=<your file name in which you want to save the cookies (this is optional)> <link of website for which you want to save the cookie sessions>**

global setup configuration:

Create a file called **global-setup.ts** and provide the following code:

```
import { chromium, expect } from "@playwright/test";

async function globalSetup() {
    const browser = await chromium.launch();
    const context = await browser.newContext();
    const page = await context.newPage();
    //Login
    await
page.goto('https://opensource-demo.orangehrmlive.com/web/index.php/dashboard/index');
    await page.getByPlaceholder('Username').fill('Admin');
    await page.getByPlaceholder('Password').fill('admin123');
    await page.getByRole('button', { name: 'Login'
}).click();
    await
page.waitForURL("https://opensource-demo.orangehrmlive.com/web/index.php/dashboard/index")
    await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
    await page.context().storageState({path:
"./playwright/.auth/auth.json"})
}
```

```
}  
  
export default globalSetup;
```

Explanation:

This is a Playwright global setup file written in TypeScript. Here is what it does, broken down:

What is Global Setup?

In Playwright, `globalSetup` runs once before all tests begin. This file is typically registered in `playwright.config.ts` and is used to handle things like authentication that you want done just once — not repeated for every single test.

Step-by-step breakdown:

```
const browser = await chromium.launch();  
const context = await browser.newContext();  
const page = await context.newPage();
```

Launches a Chromium browser, creates a fresh browser context (like a new incognito session), and opens a blank page.

```
await page.goto('https://opensource-demo.orangehrmlive.com/...');  
await page.getByPlaceholder('Username').fill('Admin');  
await page.getByPlaceholder('Password').fill('admin123');  
await page.getByRole('button', { name: 'Login' }).click();
```

Navigates to the OrangeHRM demo app, fills in the username and password fields, and clicks the Login button.

```
await page.waitForURL("https://...dashboard/index")  
await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
```

Waits for the page to redirect to the dashboard URL after login, then asserts that the Dashboard heading is visible — confirming login was successful.

```
await page.context().storageState({ path: "./playwright/.auth/auth.json" })
```

This is the most important line. It saves the authenticated browser state — cookies, localStorage, sessionStorage — to a JSON file. Your actual tests can then load this saved state and skip the login step entirely, making the test suite significantly faster.

The overall flow in one sentence:

Log in once → verify success → save the session to disk → all tests reuse that session without logging in again.

Typical usage in `playwright.config.ts`:

```
globalSetup: require.resolve('./global-setup'),
```

And in each test project:

```
storageState: './playwright/.auth/auth.json'
```

This pattern is the standard Playwright approach for handling authentication efficiently across large test suites.

Note: to execute the test use the command `npx playwright test`

Using dependencies:

Add the required dependencies under the projects section in the `playwrights.config.ts` file

- **Read test data from JSON file-** parametrize tests with different sets of data that is execute the same test with different sets of data
- In VSCode:
 1. Create a folder called **testdata**
 2. Create a file under **testdata folder** called **testdata1.json** and include the following code:

```
[
  {
    "fname": "Test1",
    "mname": "Automate1",
    "lname": "Code1",
    "email": "testcodeautomate1@gmail.com"
  },
  {
    "fname": "Test2",
    "mname": "Automate2",
    "lname": "Code2",
    "email": "testcodeautomate2@gmail.com"
  },
  {
    "fname": "ABC",
    "mname": "PQR",
    "lname": "xyz",
    "email": "abc@gmail.com"
  }
]
```

The above code contains an array of data (since there are 3 elements in the array the test will get executed 3 times)

3. Create a file under **testdata folder** called **logindata1.json** and include the following code:

```
{
  "username": "Admin",
  "password": "admin123"
}
```

4. Create a test script file under **tests folder** called **ReadDatafromExternalJSONfile.spec.ts** and include the following code:

```
import { test, expect } from "@playwright/test"
```

```

import dataArray from "../testdata/testdata1.json"
import logindata from "../testdata/logindata.json"

test.beforeEach(async ({ page }) => {
    //Login
    await
    page.goto('https://opensource-demo.orangehrmlive.com/web/index.php/dashboard/index');
    await
    page.getByPlaceholder('Username').fill(logindata.username);
    await
    page.getByPlaceholder('Password').fill(logindata.password);
    await page.getByRole('button', { name: 'Login' }).click();
    await
    page.waitForURL("https://opensource-demo.orangehrmlive.com/web/index.php/dashboard/index")
    await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
})

test("Verify timesheet card navigation on Dashboard page", async ({ page, context }) => {
    await
    expect(page.locator('#app')).toContainText('Quick Launch');
    await expect(page.getByRole('button', { name: 'Timesheets' })).toBeVisible();
    await page.getByRole('link', { name: 'Time' }).click();
    await expect(page.getByLabel('Topbar Menu').getByRole('list')).toContainText('Timesheets');
})

dataArray.forEach((data) =>{
    test(`Add Candidate for Recruitment ${data.fname}`, async ({ page }) => {
        await page.getByRole('link', { name: 'Recruitment' }).click();
    });
});

```



```

        await page.getByRole('button', { name: 'Add'
    }).click();

    await
    expect(page.locator('#app')).toContainText('Add
    Candidate');

    await    page.getByPlaceholder('First
    Name').fill(data.fname);

    await    page.getByPlaceholder('Last
    Name').fill(data.lname);

    await    page.getByPlaceholder('Middle
    Name').fill(data.mname);

    await    page.getByPlaceholder('Type
    here').first().fill(data.email);

    await page.getByRole('button', { name: 'Save'
    }).click();

    await expect(page.getByText('Application
    Stage')).toBeVisible();

    })
    })

// for(const data of dataArray){
//         test(`Add  Candidate  for  Recruitment  1
    ${data.fname}`, async ({ page }) => {
//             await page.getByRole('link', { name:
    'Recruitment' }).click();
//             await page.getByRole('button', { name: '
    Add' }).click();
//
    await
    expect(page.locator('#app')).toContainText('Add
    Candidate');

//             await    page.getByPlaceholder('First
    Name').fill(data.fname);

//             await    page.getByPlaceholder('Last
    Name').fill(data.lname);

//             await    page.getByPlaceholder('Middle
    Name').fill(data.mname);

//             await    page.getByPlaceholder('Type
    here').first().fill(data.email);

//             await page.getByRole('button', { name:
    'Save' }).click();

//             await expect(page.getByText('Application
    Stage')).toBeVisible();

```

```
//      })  
// }
```

Explanation of above code:

Here is a full breakdown of this Playwright test file:

Imports

```
import { test, expect } from "@playwright/test"  
import dataArray from "../testdata/testdata1.json"  
import logindata from "../testdata/logindata.json"
```

- **test, expect** — core Playwright testing utilities
 - **dataArray** — an array of candidate objects (fname, lname, mname, email) from a JSON file
 - **logindata** — login credentials stored externally, keeping sensitive data out of test code
-

test.beforeEach — Runs Before Every Test

```
test.beforeEach(async ({ page }) => {  
    await page.goto('...');  
    await  
    page.getByPlaceholder('Username').fill(logindata.username)  
    ;  
    await  
    page.getByPlaceholder('Password').fill(logindata.password)  
    ;  
    await page.getByRole('button', { name: 'Login' }).click();  
    await page.waitForURL("...dashboard/index")  
    await expect(page.getByRole('heading', { name:  
'Dashboard' })).toBeVisible();  
})
```

Logs in before each individual test runs. Credentials are pulled from `logindata.json` rather than hardcoded — a clean separation of test data from test logic.

Test 1 — Timesheet Navigation

```
test("Verify timesheet card navigation on Dashboard page",
  async ({ page, context }) => {
    await expect(page.locator('#app')).toContainText('Quick
    Launch');
    await expect(page.getByRole('button', { name:
    'Timesheets' })).toBeVisible();
    await page.getByRole('link', { name: 'Time' }).click();
    await expect(page.getByLabel('Topbar
    Menu').getByRole('list')).toContainText('Timesheets');
  })
```

Verifies that:

- The Quick Launch section is present on the dashboard
 - The Timesheets button is visible
 - Clicking the Time link navigates correctly and shows Timesheets in the top menu
-

Test 2 — Data-Driven Candidate Addition

```
dataArray.forEach((data) => {
  test(`Add Candidate for Recruitment ${data.fname}`,
  async ({ page }) => {
    await page.getByRole('link', { name: 'Recruitment'
    }).click();
    await page.getByRole('button', { name: 'Add' }).click();
    await expect(page.locator('#app')).toContainText('Add
    Candidate');
    await page.getByPlaceholder('First
    Name').fill(data.fname);
    await page.getByPlaceholder('Last
    Name').fill(data.lname);
    await page.getByPlaceholder('Middle
    Name').fill(data.mname);
```

```

        await page.getByPlaceholder('Type
here').first().fill(data.email);
        await page.getByRole('button', { name: 'Save' }).click();
        await expect(page.getByText('Application
Stage')).toBeVisible();
    })
})

```

This is data-driven testing. **forEach** loops over every object in **testdata1.json** and dynamically generates a separate test for each one. So if your JSON looks like:

```

[
  { "fname": "John", "lname": "Doe", "mname": "A", "email":
    "john@test.com" },
  { "fname": "Jane", "lname": "Smith", "mname": "B",
    "email": "jane@test.com" }
]

```

Playwright generates and runs two separate tests, each with its own name:

- **Add Candidate for Recruitment John**
- **Add Candidate for Recruitment Jane**

Each test navigates to Recruitment, fills the form, saves, and confirms the Application Stage screen appears.

Commented-Out **for...of** Block

```
// for(const data of dataArray){ ... }
```

This is an alternative syntax for the same data-driven loop using **for...of** instead of **forEach**. Both achieve the same result — it is commented out simply to avoid duplicate tests. The key difference:

forEach

Functional, array
method

for...of

Loop syntax, more
readable

Cannot use `await`
inside directly

Both work fine for
generating
Playwright tests

Can use `await`
safely

Both work fine for
generating
Playwright tests

The Overall Flow

beforeEach: Login



Test 1: Check dashboard timesheet navigation



Test 2a: Add candidate "John" (from testdata1.json row 1)

Test 2b: Add candidate "Jane" (from testdata1.json row 2)

... and so on for every row in the JSON

The key design pattern here is externalizing test data — credentials in `logindata.json`, candidate data in `testdata1.json` — which makes the tests reusable, maintainable, and easy to scale just by adding rows to the JSON files.

Note: In `playwright.config.ts` file if we want to run the tests parallelly we can set the parameters `fully parallel` to true and in `workers: process.env.CI ? 1 : undefined`, change the 1 to any other number like 2, 3, 4, etc to run 2 or 3 or 4 tests in parallel

Note: If a test runs for more than once because its looping through an array using `.forEach` or `for..of` method then the test can't contain the same title for all the times it runs, so its important to provide a unique value for the test each time it runs

- **Read test data from CSV file-** CSV contains comma separated values- parametrize tests with different sets of data that is execute the same test with different sets of data-the Playwright test-runner runs in Node.js which means you can directly read files from the file system and parse them with your preferred CSV library-here we are going to

use the CSV-parse package which is a parser that converts CSV text input into arrays or objects

- In VSCode:

1. Install the **csv-parse** package in your **powershell terminal** using the command **npm i csv-parse**
2. Under the **testdata** folder create a CSV file called **testdata.csv** and add the following:

```
"Id","FirstName","LastName"
"1","John","xyz"
"2","Jim","abc"
"3","Mike","pqr"
```

3. Create a new test file in the **tests** folder called **ReadDatafromExternalCSVfile.spec.ts** and add the following code:

```
import { test } from "@playwright/test";
import fs from "fs";
import { parse } from "csv-parse/sync";

const records =
parse(fs.readFileSync("testdata/testdata.csv"), {
  columns: true,
  skip_empty_lines: true,
  //delimiter: ";"
})

records.forEach((record: any) => {
  test(`Get Data from CSV forEach practice -
${record.Id}`, async ({ page }) => {
    //console.log(records);

    await
page.goto("https://demoqa.com/automation-practice-form");

    await page.getByPlaceholder('First
Name').fill(record.FirstName);
    await page.getByPlaceholder('Last
Name').fill(record.LastName);
  })
})

for (const record of records as any[]) {
```

```

        test(`Get Data from CSV for loop practice-
${record.Id}`, async ({ page }) => {
            //console.log(records);

            await
page.goto("https://demoqa.com/automation-practice-form
");

            await page.getByPlaceholder('First
Name').fill(record.FirstName);

            await page.getByPlaceholder('Last
Name').fill(record.LastName);

        })
    }
}

```

Explanation of code:

Code Explanation

This is a Playwright test file (TypeScript) that reads test data from a CSV file and runs automated browser tests for each row.

Imports & Setup

```

import { test } from "@playwright/test";
import fs from "fs";
import { parse } from "csv-parse/sync";

```

- **test** — Playwright's test runner function
- **fs** — Node's built-in file system module, used to read the CSV file
- **parse** — a synchronous CSV parser from the **csv-parse** library

Reading & Parsing the CSV

```

const records =
parse(fs.readFileSync("testdata/testdata.csv"), {
    columns: true,

```

```

        skip_empty_lines: true,
        //delimiter:";"
    })

```

- `fs.readFileSync(...)` reads the CSV file as a buffer
- `parse(...)` converts it into an array of objects, one per row
- `columns: true` means the first row is treated as headers, so each record becomes `{ Id: "1", FirstName: "John", LastName: "Doe" }`
- `skip_empty_lines: true` ignores blank rows
- The commented-out `delimiter: ";"` shows it defaults to comma (,) but could be switched to semicolon if needed

Method 1 — `forEach` Loop

```

records.forEach((record: any) => {
    test(`Get Data from CSV forEach practice - ${record.Id}`,
    async ({ page }) => {
                                                                    await
        page.goto("https://demoqa.com/automation-practice-form")
        ;
                                                                    await
        page.getByPlaceholder('First
        Name').fill(record.FirstName);
                                                                    await
        page.getByPlaceholder('Last
        Name').fill(record.LastName);
    })
})

```

- Iterates over each CSV row using `.forEach()`
- For each record, it registers a Playwright test with a dynamic name using `record.Id`
- Inside each test, it opens a demo form page and fills in the First/Last Name fields from the CSV data

Method 2 — `for...of` Loop

```

for (const record of records as any[]) {
    test(`Get Data from CSV for loop practice- ${record.Id}`,
    async ({ page }) => {

```



```

                                                                    await
page.goto("https://demoqa.com/automation-practice-form")
;
                                                                    await
page.getByPlaceholder('First
Name').fill(record.FirstName);
                                                                    await
page.getByPlaceholder('Last
Name').fill(record.LastName);
    })
}

```

- Does the exact same thing as the **forEach** block, just using a **for...of** loop instead
- **records as any[]** is a TypeScript cast since **parse()** returns an untyped result
- This is a comparison/practice demonstration of two equivalent iteration styles

Both loops produce identical behavior — if the CSV has 3 rows, you get 6 total tests (3 from **forEach** + 3 from **for...of**).

Another form of explanation:

Code Explanation

What This Code Does

This is a Playwright test automation script that reads test data from a CSV file and runs automated browser tests for each row of data.

Breaking It Down Step by Step

1. Imports

```
import { test } from "@playwright/test";
```

```
import fs from "fs";
```

```
import { parse } from "csv-parse/sync";
```

Import

Purpose

test Playwright's test runner function

fs Node.js file system — reads files from your computer

parse CSV parser that converts CSV text into JavaScript objects

2. Reading and Parsing the CSV File

```
const records = parse(fs.readFileSync("testdata/testdata.csv"), {  
  columns: true,      // First row becomes the column headers/keys  
  skip_empty_lines: true, // Ignores blank lines in the CSV  
  //delimiter:";"      // Commented out - would use semicolon instead of  
  comma  
})
```

If your CSV looks like this:

Id,FirstName,LastName

1,John,Doe

2,Jane,Smith

records becomes this:

[

```
{ Id: '1', FirstName: 'John', LastName: 'Doe' },  
{ Id: '2', FirstName: 'Jane', LastName: 'Smith' }  
]
```

3. First Loop — forEach Method

```
records.forEach((record: any) => {  
  test(`Get Data from CSV forEach practice - ${record.Id}`, async ({ page }) => {  
    await page.goto("https://demoqa.com/automation-practice-form");  
    await page.getByPlaceholder('First Name').fill(record.FirstName);  
    await page.getByPlaceholder('Last Name').fill(record.LastName);  
  })  
})
```

What happens:

- Loops through every row in the CSV
 - Creates a separate test for each row
 - Each test is named using the record's Id (e.g., **Get Data from CSV forEach practice - 1**)
 - Opens the browser, goes to the form page
 - Fills in First Name and Last Name from the CSV data
-

4. Second Loop — for...of Method

```
for (const record of records as any[]) {  
  test(`Get Data from CSV for loop practice- ${record.Id}`, async ({ page }) => {  
    await page.goto("https://demoqa.com/automation-practice-form");  
    await page.getByPlaceholder('First Name').fill(record.FirstName);  
  })  
}
```

```

    await page.getByPlaceholder('Last Name').fill(record.LastName);
  })
}

```

This does the exact same thing as the `forEach` loop — it is just a different JavaScript syntax for looping.

Key Concept — Data Driven Testing

This is called Data Driven Testing — instead of hardcoding test values, you pull them from an external source (CSV file), so:

CSV Row 1 → Creates Test 1 → Fills form with Row 1 data

CSV Row 2 → Creates Test 2 → Fills form with Row 2 data

CSV Row 3 → Creates Test 3 → Fills form with Row 3 data

forEach vs for...of — What is the Difference?

	forEach	for...of
Style	Functional/callback	Traditional loop
Behavior here	Identical	Identical
Can break/continue	No	Yes

Readability

Clean for simple loops

Better for complex logic

Visual Flow Summary

CSV File



`fs.readFileSync()` — reads the file



`parse()` — converts CSV to array of objects



Loop through each record



Create a Playwright test per record



Open browser → Go to form → Fill First Name → Fill Last Name

In short — 2 CSV rows = 4 total tests (2 from `forEach` + 2 from `for...of`), each filling the form with different data automatically.

- **Pass test data as fixtures-** when we pass test data as fixtures it only supports a single set of data-hence we cannot parameterize our test that is we cannot run/execute the same test with different sets of data
- In VSCode:
 1. Under the **fixtures** folder create a file called **TestDataFixture.ts** and include the following code:

```
import { test as baseTest } from "@playwright/test";  
  
type MyFixture = {
```

```

    logindata: any;
    testdata: any;
  }

export const test = baseTest.extend<MyFixture>({
  logindata: {
    uname: "Admin",
    pwd: "admin123",
  },
  testdata: {
    fname: "Sam",
    mname: "D",
    lname: "Warn",
    email: "same@gmail.com"
  }
})

export { expect } from "@playwright/test"

```

Explanation of code:

Code Explanation

This file defines custom Playwright fixtures — a way to inject reusable test data into your tests automatically.

Base Import

```
import { test as baseTest } from "@playwright/test";
```

- Imports Playwright's built-in **test** object but aliases it as **baseTest**
- This is done so you can extend it without naming conflicts

Type Definition

```
type MyFixture = {
```

```

    logindata: any;
    testdata: any;
  }

```

- Defines a TypeScript type that describes the shape of your custom fixtures
- **logindata** — will hold login credentials
- **testdata** — will hold form/user data
- Both are typed as **any** for flexibility (could be more strictly typed in production)

Extending the Base Test

```

export const test = baseTest.extend<MyFixture>({
  logindata: {
    uname: "Admin",
    pwd: "admin123",
  },
  testdata: {
    fname: "Sam",
    mname: "D",
    lname: "Warn",
    email: "same@gmail.com"
  }
})

```

- **baseTest.extend<MyFixture>(...)** creates a new test object with your custom fixtures baked in
- **<MyFixture>** is the generic type parameter that tells Playwright what fixtures to expect
- The two fixtures defined are:

Pr
op
ert
ies

Values

```
un      "Admin",
am      "admin123
e,      "
pw
d
```

```
fn      "Sam", "D",
am      "Warn",
e,      "same@gma
mn      il.com"
am
e,
ln
am
e,
em
ai
l
```

- This **test** is then exported so other test files can import and use it

Re-exporting **expect**

```
export { expect } from "@playwright/test"
```

- Re-exports Playwright's **expect** assertion library
- This means test files only need one import to get both **test** and **expect**:

```
import { test, expect } from "../fixtures"; // one import covers
everything
```

How It's Used in a Test File

```
import { test, expect } from "../fixtures";

test("Login test", async ({ page, logindata, testdata }) => {
  // logindata and testdata are automatically available
  console.log(logindata.uname); // "Admin"
  console.log(testdata.fname);  // "Sam"
});
```

Key Takeaway

Detail

Centralizes test data in one reusable place

No need to repeat data in every test file

Fixture-based dependency injection

These are plain object fixtures (no **async** setup/teardown needed)

Think of fixtures as automatic parameters — Playwright injects them into every test that requests them, keeping your tests clean and DRY (Don't Repeat Yourself).

2. Under the **tests** folder create a file called **TestDatafromFixture.spec.ts** and include the following code:

```
import {test, expect} from
"../fixtures/TestDataFixture"

test.beforeEach("Login", async ({page, logindata}) => {
    //Login
    await
    page.goto('https://opensource-demo.orangehrmlive.com/w
eb/index.php/auth/login');
    await
    page.getByPlaceholder('Username').fill(logindata.uname
);
    await
    page.getByPlaceholder('Password').fill(logindata.pwd);
    await page.getByRole('button', { name: 'Login'
}).click();
    await expect(page.getByRole('heading', { name:
'Dashboard' })).toBeVisible();
})

test('Add Candidate for Recruitment', async ({ page,
testdata }) => {
    await page.getByRole('link', { name: 'Recruitment'
}).click();
    await page.getByRole('button', { name: ' Add'
}).click();
    await
    expect(page.locator('#app')).toContainText('Add
Candidate');
```

```

        await page.getByPlaceholder('First
Name').fill(testdata.fname);
        await page.getByPlaceholder('Middle
Name').fill(testdata.mname);
        await page.getByPlaceholder('Last
Name').fill(testdata.lname);
        await page.getByPlaceholder('Type
here').first().fill(testdata.email);
        await page.getByRole('button', { name: 'Save'
}).click();
        await expect(page.getByText('Application
Stage')).toBeVisible();
    })

```

Explanation of code:

Code Explanation

This is a Playwright test file that uses the custom fixtures defined earlier to test the "Add Candidate" feature on a demo HR application.

Import

```
import { test, expect } from "../fixtures/TestDataFixture"
```

- Imports the custom **test** (with fixtures pre-loaded) and **expect** from the fixture file
- Because of this single import, **logindata** and **testdata** are automatically available in every test

test.beforeEach — Login Hook

```

test.beforeEach("Login", async ({ page, logindata }) => {
    await
    page.goto('https://opensource-demo.orangehrmlive.com/web/index.php/auth/login');
    await
    page.getByPlaceholder('Username').fill(logindata.uname);

```

```

                                                                    await
page.getByPlaceholder('Password').fill(logindata.pwd);
  await page.getByRole('button', { name: 'Login' }).click();
    await expect(page.getByRole('heading', { name:
'Dashboard' })).toBeVisible();
  })

```

Part	Detail
test .beforeEach	Runs before every test in this file
"Login"	A label/description for this hook
{ page , logindata }	Destructures Playwright's page + your custom logindata fixture
page .goto('')	Navigates to the OrangeHRM login page
.fill(logindata.username/pwd)	Fills credentials from fixture — "Admin" / "admin123"
.click()	Clicks the Login button

```
expect(
  page.getByRole('link', {
    name: 'Recruitment'
  }).click()
).toBeVisible();
```

Assertion — verifies the Dashboard heading appears, confirming login succeeded

Every test in this file will be pre-logged-in before it runs.

The Test — Add Candidate for Recruitment

```
test('Add Candidate for Recruitment', async ({ page, testdata }) => {
```

- `testdata` fixture is injected automatically — contains `fname`, `mname`, `lname`, `email`

Step-by-Step Breakdown

```
await page.getByRole('link', { name: 'Recruitment' }).click();
```

- Clicks the Recruitment link in the nav menu

```
await page.getByRole('button', { name: 'Add' }).click();
```

- Clicks the Add button to open the candidate form

```
await expect(page.locator('#app')).toContainText('Add Candidate');
```

- Assertion — verifies the page now shows "Add Candidate", confirming navigation worked

```
await page.getByPlaceholder('First Name').fill(testdata.fname); // "Sam"
await page.getByPlaceholder('Middle Name').fill(testdata.mname); // "D"
```

```
await page.getByPlaceholder('Last Name').fill(testdata.lname); // "Warn"
await page.getByPlaceholder('Type here').first().fill(testdata.email); // "same@gmail.com"
```

- Fills the candidate form using data from the **testdata** fixture
- **.first()** is used on email because there may be multiple "Type here" placeholders on the page

```
await page.getByRole('button', { name: 'Save' }).click();
```

- Submits the form

```
await expect(page.getByText('Application Stage')).toBeVisible();
```

- Final assertion — verifies "Application Stage" appears, confirming the candidate was saved successfully

Full Test Flow Visualized

beforeEach runs first



Navigate to Login Page



Fill Username & Password (from logindata fixture)



Click Login → Assert Dashboard visible



[Test starts]

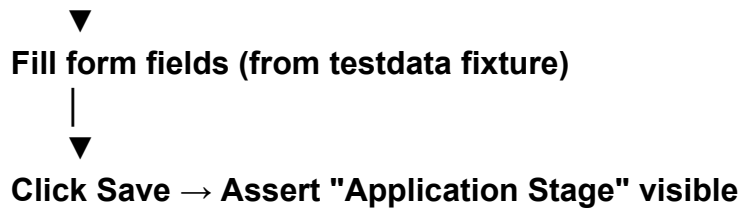


Click Recruitment → Click Add



Assert "Add Candidate" page loaded





Perform Mouse Hover action in Playwright | Playwright Tutorials

To Hover over the matching element we need to use the hover method.
`await locator.hover();`

This method hovers over the element by performing the following steps:

- Wait for actionability checks on the element (it should be Visible, Stable and Receives Events), unless the force option is set.
- Scroll the element into view if needed.
- Use `page.mouse` to hover over the centre of the element, or the specified position.
- Wait for initiated navigations to either succeed or fail.

In VSCode:

1. Create a file called **PracticeMouseHover.spec.ts** under the **tests** folder and include the following code:

```
import {test} from "@playwright/test";

test("Practice Mouse hover action", async({page})=>{
    await page.goto('https://demo.opencart.com/');
    await page.getByRole('link', { name:
'Components', exact: true }).hover({force:true});
    await page.getByRole('link', { name: 'Show All
Components' }).click()
})
```

Explanation of code:

Code Explanation

This is a Playwright test that demonstrates mouse hover interaction to reveal and interact with a dropdown menu.

Import

```
import { test } from "@playwright/test";
```

- Imports only **test** from Playwright — no **expect** needed since there are no assertions in this test (it's purely a practice/interaction test)
-

Test Definition

```
test("Practice Mouse hover action", async ({ page }) => {
```

- Defines a test named "**Practice Mouse hover action**"
 - page** is Playwright's built-in browser page fixture
-

Step-by-Step Breakdown

Step 1 — Navigate to the site

```
await page.goto('https://demo.opencart.com/');
```

- Opens the OpenCart demo e-commerce website
 - Waits until the page is fully loaded before moving on
-

Step 2 — Hover over the "Components" menu

```
await page.getByRole('link', { name: 'Components', exact: true }).hover({ force: true });
```

P
a
r
t

Detail

g
e
t
B
y

Finds an element by its ARIA role —
in this case a **<a>** link

**R
o
l
e
(
l
i
n
k
,
,
.
.
.
)
n
a
m
e
:
,
C
o
m
p
o
n
e
n
t
s
,

e
x
a**

**Matches the link with the text
"Components"**

**Ensures exact text match — won't
match "Components & More" or "All
Components"**

```
c  
t  
:  
t  
r  
u  
e  
  
.h  
o  
v  
e  
r  
(  
)  
  
{  
f  
o  
r  
c  
e  
:  
t  
r  
u  
e  
}
```

Simulates a mouse hover over the element

Forces the hover even if the element is hidden, overlapped, or not actionable

Hovering over "Components" triggers a dropdown menu to appear, which contains sub-links like "Show All Components"

Step 3 — Click the revealed dropdown item
await page.getByRole('link', { name: 'Show All Components' }).click();

- After the hover reveals the dropdown, this clicks the "Show All Components" link inside it
- Without the hover in Step 2, this link would be invisible/inaccessible and the click would fail

Why **force: true** is Used

Normally Playwright checks that an element is:

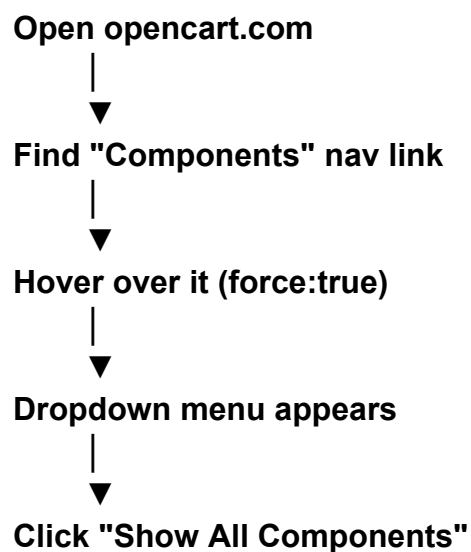
- Visible
- Not overlapped
- Stable (not animating)

On some sites, dropdown trigger elements can be partially hidden or intercepted by other elements. **force: true** bypasses these checks:

Without force:true	With force:true
Playwright checks actionability → may throw error	Playwright skips actionability → hover fires directly

⚠ Use **force: true** carefully — it can mask real UI issues. Only use it when you're certain the element exists but has actionability constraints.

Hover Flow Visualized





Navigate to Components page

force Option & Actionability in Playwright

What is Actionability?

Before Playwright performs any action (click, hover, fill, etc.), it automatically runs a set of actionability checks to ensure the element is ready to interact with. This mimics how a real user would interact with a page.

Actionability Checks by Action

Different actions have different checks:

What Each Check Means

1. Visible

// Element must be visible — not hidden by CSS

display: none // ✗ fails

visibility: hidden // ✗ fails

opacity: 0 // ✗ fails

normal element // ✓ passes

2. Stable

// Element must NOT be animating or moving

// Playwright waits for the element to stop moving before acting

<div class="animating-dropdown"> // ✗ waits...

<div class="fully-loaded"> // ✓ passes

3. Enabled

// Element must not be disabled

<button disabled>Submit</button> // ✗ fails

```
<button>Submit</button> //  passes
```

4. Editable

// Element must be editable (for fill)

```
<input readonly /> //  fails
```

```
<input /> //  passes
```

5. Receives Events

// Element must not be covered/overlapped by another element

// i.e. a real mouse click would actually hit THIS element

```
<button style="z-index:-1"> //  covered by overlay
```

```
<button> //  receives events
```

What Does **force: true** Do?

// Without force — all actionability checks run

```
await page.getByRole('link', { name: 'Components' }).hover();
```

// With force — ALL actionability checks are SKIPPED

```
await page.getByRole('link', { name: 'Components' }).hover({ force: true });
```

force: true tells Playwright:

"Don't check if the element is ready — just perform the action immediately"

When to Use **force: true**

Valid Use Cases

// 1. Element is visually hidden but still in the DOM

```
await page.getByRole('menu').click({ force: true });
```

// 2. Custom dropdown triggers that are partially obscured

```
await page.locator('.nav-item').hover({ force: true });
```

```
// 3. Element is covered by a transparent overlay
await page.getByRole('button').click({ force: true });
```

✗ When NOT to Use It

```
// Don't use force just to silence errors without
// understanding why
// It can hide real bugs in your application
```

```
await page.getByRole('button', {name: 'Submit'}).click({
  force: true });
// If button is disabled, force will click it anyway — masking
// a real issue!
```

force vs Waiting

Normal Action

force: true

Check visible	→ fail?	Skip all checks
└─ wait & retry		
Check stable	→ fail?	▼
└─ wait & retry		Dispatch event directly
Check enabled	→ fail?	on the DOM element
└─ throw error		
Check receives events	→ fail?	
└─ throw error		

Key Takeaways

Detail

Playwright's built-in safety
checks before every
action

Ensures interactions
behave like a real user

Skips ALL actionability
checks

Can mask real UI bugs —
use sparingly

Prefer fixing the root
cause over using **force:**
true

Think of actionability checks as Playwright asking "Would a real user be able to do this right now?" — **force: true** bypasses that question entirely.

Drag & Drop Action in Playwright | Playwright Tutorials

Drag the source element towards the target element and drop it.

Approach 1 - Dragging manually, If you want precise control over the drag operation, use lower-level methods like `locator.hover()`, `mouse.down()` and `mouse.up()`.

Approach 2 - You can perform drag&drop operation with `locator.dragTo()`. This method will:

- Hover the element that will be dragged.
- Press the left mouse button.
- Move the mouse to the element that will receive the drop.
- Release left mouse button.

// or

to specify exact positions relative to the top-left corners of the elements:

```
await source.dragTo(target, {
  sourcePosition: { x: 34, y: 7 },
  targetPosition: { x: 10, y: 20 },
```

In VSCode:

1. Create a file called **PracticeDragandDrop.spec.ts** under the **tests** folder and include the following code:

```
import {test, expect} from "@playwright/test";

test("Practice Drag and Drop", async({page})=>{

  await page.goto('https://demoqa.com/droppable');
```



```

// Approach 1 - Manual way to Drag and Drop

// await page.getByText('Drag me', { exact: true }).hover();

// await page.mouse.down();

// await
page.getByLabel('Simple').locator('#draggable').hover();

// await page.mouse.up();

// await
expect(page.getByLabel('Simple').locator('#draggable')).toHaveText("Dropped!")

//Approach 2 - dragTo()

    await page.getByText('Drag me', { exact: true
}).dragTo(page.getByLabel('Simple').locator('#draggable'), {

    sourcePosition:{x:0,y:0},

    targetPosition:{x:149.9,y:149.9}

})

//await
expect(page.getByLabel('Simple').locator('#draggable')).toHaveText("Dropped!")

})

```

Explanation of code:

Code Explanation

This Playwright test demonstrates two approaches to Drag and Drop interaction on a demo page.

Import & Setup

```
import { test, expect } from "@playwright/test";
```

```
test("Practice Drag and Drop", async ({ page }) => {
```

```
    await page.goto('https://demoqa.com/droppable');
```

- Navigates to DemoQA's droppable practice page
 - The page has a draggable element and a droppable target zone
-

Approach 1 — Manual Drag and Drop (Commented Out)

```
await page.getByText('Drag me', { exact: true }).hover();
```

```
await page.mouse.down();
```

```
await page.getByLabel('Simple').locator('#droppable').hover();
```

```
await page.mouse.up();
```

```
await  
expect(page.getByLabel('Simple').locator('#droppable')).toHaveText("Dropped!")
```

This manually simulates what a real mouse does during drag and drop:

Code

What it does

`.hover()`

Moves mouse cursor onto the draggable element

`mouse.down()`

Presses and holds the left mouse button

`.hover()` on target

Moves the mouse (while held) to the drop zone

`mouse.up()`

Releases the mouse button, completing the drop

`expect(...).toHaveText("Dropped!")`

Asserts the drop zone text changed to confirm success

Visual Flow

Hover over "Drag me"

|



`mouse.down()` ← hold button

|



Hover over drop target ← drag

|



`mouse.up() ← release = drop`

|



Assert "Dropped!" text appears

Approach 2 — `dragTo()` Method (Active)

```
await page.getByText('Drag me', { exact: true }).dragTo(
  page.getByLabel('Simple').locator('#draggable'),
  {
    sourcePosition: { x: 0, y: 0 },
    targetPosition: { x: 149.9, y: 149.9 }
  }
)
```

This uses Playwright's built-in `dragTo()` which handles the entire drag and drop in one call.

Breaking Down `dragTo()`

The Source (what to drag)

```
page.getByText('Drag me', { exact: true })
```

- Locates the draggable element by its exact text "Drag me"
- `exact: true` prevents matching "Drag me somewhere" etc.

The Target (where to drop)

```
page.getByLabel('Simple').locator('#draggable')
```

- `getByLabel('Simple')` — finds the "Simple" tab/section
- `.locator('#draggable')` — finds the drop zone inside that section
- Chaining is used because there are multiple `#draggable` elements on the page (Simple, Accept, Prevent tabs)

The Options

```
{  
  sourcePosition: { x: 0, y: 0 },    // grab point on draggable  
  targetPosition: { x: 149.9, y: 149.9 } // release point on droppable  
}
```

Option

Description

`sourcePosition`

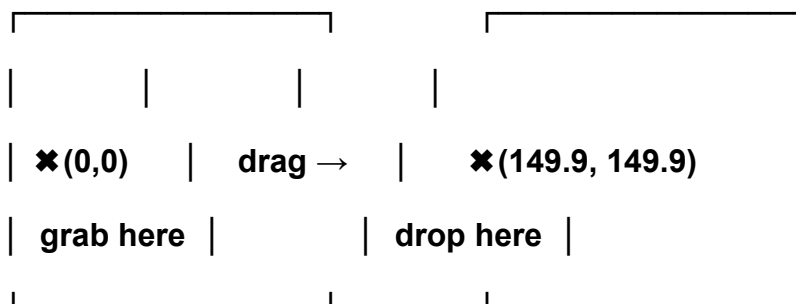
The `x,y` coordinates within the draggable element where the drag starts

`targetPosition`

The `x,y` coordinates within the drop zone where the element is released

Draggable Element

Drop Zone



sourcePosition

targetPosition

Approach 1 vs Approach 2 Comparison

Aspect	Approach 1 (Manual)	Approach 2 (dragTo)
Code length	4 steps	1 step
Readability	Verbose	Clean
Control	Full mouse control	Limited to positions
Reliability	Can be flaky	More stable
Use case	Complex drag scenarios	Standard drag and drop
Built-in	Raw mouse API	Playwright helper method

Why `sourcePosition` and `targetPosition` Matter

// Without positions — Playwright uses element centers by default

```
await source.dragTo(target);
```

// With positions — you control exact grab and drop coordinates

```
await source.dragTo(target, {  
  sourcePosition: { x: 0, y: 0 },    // top-left corner of source  
  targetPosition: { x: 149.9, y: 149.9 } // near center of target  
});
```

- Some drag-and-drop implementations are position-sensitive — they only work if you grab/drop at specific coordinates
- `149.9` is used instead of `150` — a common trick to avoid pixel boundary issues on some browsers

Key Takeaways

Concept	Detail
Manual approach	Uses raw <code>mouse.down()</code> / <code>mouse.up()</code> for full control
<code>dragTo()</code>	Playwright's clean, built-in drag and drop method
<code>sourcePosition</code>	Where on the draggable element to start the drag

targetPosition

Where on the drop zone to release the drag

Locator chaining

Used to disambiguate between multiple similar elements

Commented assertion

expect("Dropped!") would verify success but is disabled here

How to Get the **targetPosition** Coordinates

There are several ways to find the exact x,y coordinates for drag and drop positions.

Method 1 — Playwright Inspector (Most Reliable)

Run in inspector/debug mode

`npx playwright test --debug`

- Opens the Playwright Inspector
- Hover over any element and the inspector shows its exact dimensions and position
- You can read the element's width/height and calculate the center or any point within it

Method 2 — Browser DevTools (Quick & Easy)

Step 1 — Open DevTools in Chrome

F12 or **Right Click** → **Inspect**

Step 2 — Use the Console to get element coordinates


```
// Get element's size and position

const el = document.querySelector('#draggable');

const rect = el.getBoundingClientRect();

console.log(rect);

// Output:

// {
//   x: 100,
//   y: 200,
//   width: 300,    ← element is 300px wide
//   height: 300,   ← element is 300px tall
//   top: 200,
//   left: 100,
//   right: 400,
//   bottom: 500
// }
```

Step 3 — Calculate your desired position

```
// Get center point

const centerX = rect.width / 2; // 300 / 2 = 150

const centerY = rect.height / 2; // 300 / 2 = 150

// Playwright uses position RELATIVE to element (not page)

// So center = (150, 150) → use 149.9 to avoid boundary issues
```

Method 3 — Playwright's `boundingBox()`

```
test("Get coordinates", async ({ page }) => {

  await page.goto('https://demoqa.com/droppable');

  // Get the bounding box of the drop target

  const dropTarget = page.getByLabel('Simple').locator('#draggable');

  const box = await dropTarget.boundingBox();

  console.log(box);

  // Output:

  // {
  //   x: 100,    ← position on page
  //   y: 200,    ← position on page
  //   width: 300, ← element width
  //   height: 300 ← element height
  // }

  // Calculate center position WITHIN the element

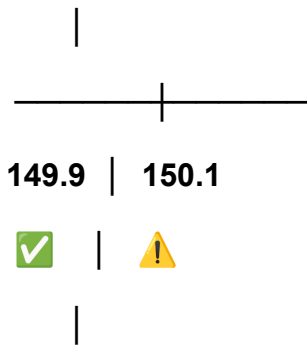
  const centerX = box.width / 2; // 150

  const centerY = box.height / 2; // 150

  })
```

Why 149.9 Instead of 150?

Element boundary



exact center

Risk

May land exactly on the border — can cause inconsistent behavior

Lands just inside the element — more reliable across browsers

Top-left corner of the element

It's a defensive practice to avoid pixel boundary edge cases.

Method 4 — Hover & Read Coordinates with Mouse Event

// Paste in browser console

```
document.querySelector('#draggable').addEventListener('mousemove',  
(e) => {  
  
    const rect = e.target.getBoundingClientRect();  
  
    // Position relative to the element itself  
  
    console.log({  
  
        relativeX: e.clientX - rect.left,  
  
        relativeY: e.clientY - rect.top  
  
    });  
  
});
```

- Move your mouse over the element
- Console prints real-time relative coordinates as you move
- Stop when you find the position you want to use

Quick Reference — Common Position Values

// Given an element with width: 300, height: 300

```
{ x: 0,    y: 0    } // Top-left corner  
  
{ x: 149.9, y: 0    } // Top-center  
  
{ x: 149.9, y: 149.9 } // Center (most common)  
  
{ x: 0,    y: 149.9 } // Middle-left  
  
{ x: 299.9, y: 299.9 } // Bottom-right corner
```

Key Takeaways

Method	Best For
Playwright Inspector	Debugging during test development
DevTools Console	Quick coordinate lookup
<code>boundingBox()</code>	Dynamic coordinate calculation in tests
Mouse event listener	Finding exact interactive positions
149.9 rule	Avoiding pixel boundary issues near center

Best practice — use `boundingBox()` in your actual tests so coordinates are calculated dynamically rather than hardcoded, making tests more resilient to layout changes.

Keyboard Actions in Playwright | Playwright Tutorials

In Playwright, the `press()` method is used to simulate keyboard actions such as pressing single or multiple combinations of keys.

Examples of Keys are -

- F1 - F12, Digit0- Digit9, KeyA- KeyZ, Backquote, Minus, Equal, Backslash, Backspace, Tab, Delete, Escape, ArrowDown, End, Enter, Home, Insert, PageDown, PageUp, ArrowRight, ArrowUp, etc.
- Following modification shortcuts are also supported: Shift, Control, Alt, Meta, ShiftLeft.
- If the key is a single character, it is case-sensitive, so the values a and A will generate different respective texts.

Syntax -

```
await page.keyboard.press(<key>);
```

```
await page.locator.press(<key>);
```

Note: The second syntax is more preferred-

await page.locator.press(<key>)

In VSCode:

Under the tests folder create a file called PracticeKeyboardActions.spec.ts and include the below code:

```
import {test} from "@playwright/test";

test("Practice Keyboard Actions", async({page})=>{
  await
  page.goto('https://testpages.eviltester.com/styled/basic-html-form-test.html');

  const commentTextArea = page.locator("[name='comments']");

  await commentTextArea.press("Control+a");

  await commentTextArea.press("Backspace");

  await commentTextArea.press("a+B+c");

  await commentTextArea.press("Control+a+x")

  const usernameInput = page.locator("[name='username']");

  await usernameInput.press("Control+v");

  await usernameInput.press("ArrowLeft+ArrowLeft+ArrowLeft");

  await usernameInput.press("Z")

  await page.keyboard.press("PageDown");

  await page.keyboard.press("PageUp");

})
```

Explanation of code:

Code Explanation

This Playwright test demonstrates various keyboard actions and shortcuts on a web form.

Import & Navigation

```
import { test } from "@playwright/test";
```

```
await  
page.goto('https://testpages.eviltester.com/styled/basic-html-form-test.html');
```

- Navigates to a basic HTML form practice page
- No **expect** imported since this is purely a keyboard interaction practice test

Two Ways to Press Keys in Playwright

element.press()	page.keyboard.press()
_____	_____
Targets a SPECIFIC	Global keypress —
element	no specific element
Must focus element first	Acts on focused element

Comment TextArea Actions

```
const commentTextArea = page.locator("[name='comments']");
```

- Locates the textarea by its **name** attribute
-

1. Select All

```
await commentTextArea.press("Control+a");
```

- **Control+a** — selects all text inside the textarea
 - **+** between keys means they are pressed simultaneously
-

2. Delete Selected Text

```
await commentTextArea.press("Backspace");
```

- After selecting all with **Ctrl+A**, **Backspace** deletes all selected text
 - Clears the textarea completely
-

3. Type Multiple Keys

```
await commentTextArea.press("a+B+c");
```

- Presses **a**, **B**, and **c** simultaneously
-  This does not type "aBc" as a sequence
- It fires all three keys at the same time as a combo
- For typing text sequentially, use **.fill()** or **.type()** instead:

```
await commentTextArea.fill("aBc"); // correct way to type text
```

4. Select All + Cut


```
await commentTextArea.press("Control+a+x");
```

- Attempts to press **Ctrl**, **a**, and **x** simultaneously
- Intended effect: Select All (**Ctrl+A**) then Cut (**Ctrl+X**)
- ⚠ This is not how chained shortcuts work — this fires all three at once
- Correct way to do select-all then cut:

```
await commentTextArea.press("Control+a"); // Step 1: Select all
```

```
await commentTextArea.press("Control+x"); // Step 2: Cut
```

Username Input Actions

```
const usernameInput = page.locator("[name='username']");
```

- Locates the username input field by its **name** attribute

5. Paste

```
await usernameInput.press("Control+v");
```

- Pastes whatever is in the clipboard into the username field
- Works with previously cut/copied content (from **Ctrl+X** above)

6. Move Cursor with Arrow Keys

```
await usernameInput.press("ArrowLeft+ArrowLeft+ArrowLeft");
```

- ⚠ Presses all three **ArrowLeft** keys simultaneously — not sequentially
- Intended to move cursor 3 positions to the left
- Correct way to move cursor left 3 times:

```
await usernameInput.press("ArrowLeft");
```

```
await usernameInput.press("ArrowLeft");
```

```
await usernameInput.press("ArrowLeft");
```

7. Type a Single Character

```
await usernameInput.press("Z");
```

- Presses the **Z** key — types the letter "Z" at the current cursor position
- Since cursor was moved left, "Z" inserts in the middle of existing text

Global Page Keyboard Actions

```
await page.keyboard.press("PageDown");
```

```
await page.keyboard.press("PageUp");
```

- These act on the entire page, not a specific element
- **PageDown** — scrolls the page down one viewport height
- **PageUp** — scrolls the page back up one viewport height
- Uses **page.keyboard** instead of **element.press()** since scrolling is a global action

Common Key Names Reference

K e y	Playwright String
-------------	-------------------

E n	"Enter"
--------	----------------

t
e
r

T
a
b

"Tab"

E
s
c
a
p
e

"Escape"

B
a
c
k
s
p
a
c
e

"Backspace"

D
e
l
e
t
e

"Delete"

A
r
r
o
w

"ArrowUp" "ArrowDown"
"ArrowLeft" "ArrowRight"

k
e
y
s

P
a
g
e
s
c
r
o
l
l

"PageUp" "PageDown"

C
t
r
l
+
C

"Control+c"

C
t
r
l
+
V

"Control+v"

C
t
r
l
+
X

"Control+x"

C	"Control+a"
t	
r	
l	
+	
A	

C	"Control+z"
t	
r	
l	
+	
Z	

press() vs fill() vs type()

Method	Use For	Example
--------	---------	---------

.press()	Single keys or shortcuts	press("Control+a")
----------	--------------------------	--------------------

.fill()	Replacing entire input value	fill("Hell")
---------	------------------------------	--------------

<code>innerHTML</code>		<code>oWorld")</code>
<code>.type()</code>	Typing character by character (simulates real typing)	<code>type("Hello")</code>

Key Takeaways

Concept	Detail
<code>+ in press()</code>	Means simultaneous keypresses, not sequential
<code>element.press()</code>	Keyboard action on a specific focused element
<code>page.keyboard.press()</code>	Global keyboard action on the whole page

**Arrow
keys**

Move cursor within input fields

**PageDown/Page
Up**

Scroll the entire page viewport


**Common
mistake**

**Using `+` to chain sequential shortcuts instead
of separate `.press()` calls**

Upload File / Files using Playwright | Playwright Tutorials

- We can select input file/files for upload using the `locator.setInputFiles()` method.
- `locator.setInputFiles()` will upload file / files on input element with the type as file.
- To upload multiple files we need to pass them as an array to the `setInputFiles()` method.
- To clear the uploaded files we need to pass an empty array to the `setInputFiles()` method.
- If you don't have an input element in hand (it is created dynamically), by using a corresponding waiting method upon action to upload file/files..

Syntax -

Single File Upload -

```
locator.setInputFiles("FileNameWithPath");
```

Upload Multiple Files -

```
locator.setInputFiles(["File1Namewithpath", "File3Namewithpath", "File2Namewithpath" .....])
```

In VSCode:

Create a file called `PracticeUploadFile.spec.ts` under the tests folder and include the following code:

```
import {test} from "@playwright/test"

import path from "path";

test("Practice File Upload", async ({page}) =>{
```

```

                                                                    await
page.goto("https://davidwalsh.name/demo/multiple-file-upload.php");

                                                                    //await
page.locator("#filesToUpload").setInputFiles("to-upload/File1.txt") //To Upload Single File

                                                                    await
page.locator("#filesToUpload").setInputFiles(["to-upload/File1.txt", "to-upload/File2.xlsx", "to-upload\\File3.docx"]) // To Upload mulipl files

    await page.locator("#filesToUpload").setInputFiles([]);
})

test("Practice File Upload 2", async({page})=>{

                                                                    await
page.goto("https://the-internet.herokuapp.com/upload");

    const fileChooserPromise =
page.waitForEvent("filechooser");

    await page.locator("#drag-drop-upload").click();

    const fileChooserResolved = await fileChooserPromise;

                                                                    await
fileChooserResolved.setFiles(["to-upload/File1.txt", "to-upload/File2.xlsx", "to-upload\\File3.docx"])

})

```

Explanation of code:

Note: Always use '/' or '\\' while including the filepath

Code Explanation

This file demonstrates two different approaches to file upload in Playwright.

Import & Setup

```
import { test } from "@playwright/test"
```

```
import path from "path";
```

- `path` is imported but not used in this example — typically used to build absolute file paths like:

```
path.join(__dirname, 'to-upload', 'File1.txt') // safer cross-platform paths
```

Test 1 — Direct Input File Upload

```
test("Practice File Upload", async ({ page }) => {
```

```
    // await
    page.goto("https://davidwalsh.name/demo/multiple-file-upload.php")
    ;
```

- Navigates to a page with a standard `<input type="file">` element
-

Single File Upload (Commented Out)

```
// await
page.locator("#filesToUpload").setInputFiles("to-upload/File1.txt")
```

- Uses `setInputFiles()` with a single string path to upload one file
 - `#filesToUpload` is the file input element's ID
-

Multiple File Upload

```
await page.locator("#filesToUpload").setInputFiles([  
    "to-upload/File1.txt",  
    "to-upload/File2.xlsx",  
    "to-upload\\File3.docx"  
])
```

- Passes an array of file paths to upload multiple files at once
- Notice two different path separator styles:

Path	Se pa rat or	Platfor m
"to-uplo ad/File1 .txt"	Fo rw ar d sl as h / 	Unix/ Mac/W indow s
"to-uplo ad/File2 .xlsx"	Fo rw ar d sl as h / 	Unix/ Mac/W indow s

`"to-upload\\File3.docx"`

Backslash

Windows only

⚠ Forward slash `/` works on all platforms — prefer it over `\\` for cross-platform compatibility

Clear Uploaded Files

```
await page.locator("#filesToUpload").setInputFiles([]);
```

- Passes an empty array to remove/clear all selected files
 - Resets the file input back to its empty state
-

Test 1 Flow

Navigate to page

|



Locate #filesToUpload input

|



```
setInputFiles([file1, file2, file3])
```

|



All 3 files selected



`setInputFiles([])` → files cleared

Test 2 — File Chooser Dialog Upload

```
test("Practice File Upload 2", async ({ page }) => {  
  await page.goto("https://the-internet.herokuapp.com/upload");
```

- Navigates to a page with a drag-and-drop / click-to-upload zone
 - This type doesn't use a standard visible `<input type="file">` — it opens a file chooser dialog
-

Step 1 — Set Up the File Chooser Listener FIRST

```
const fileChooserPromise = page.waitForEvent("filechooser");
```

- Creates a Promise that waits for the file chooser dialog to open
- ⚠ This **MUST** be set up before clicking — if you click first, the dialog opens and closes before the listener is ready:

✅ Correct Order ❌ Wrong Order

- | | |
|--------------------------------|--------------------------------|
| 1. <code>waitForEvent()</code> | 1. <code>click()</code> |
| 2. <code>click()</code> | 2. <code>waitForEvent()</code> |
| 3. <code>await promise</code> | 3. Dialog already gone! |

Promise never resolves

Step 2 — Click to Trigger the Dialog

```
await page.locator("#drag-drop-upload").click();
```

- Clicking the drag-drop zone opens the OS file chooser dialog
- The **fileChooserPromise** catches this event

Step 3 — Resolve the Promise

```
const fileChooserResolved = await fileChooserPromise;
```

- **await** waits for the file chooser dialog to actually open
- Returns a FileChooser object that lets you interact with the dialog

Step 4 — Select Files

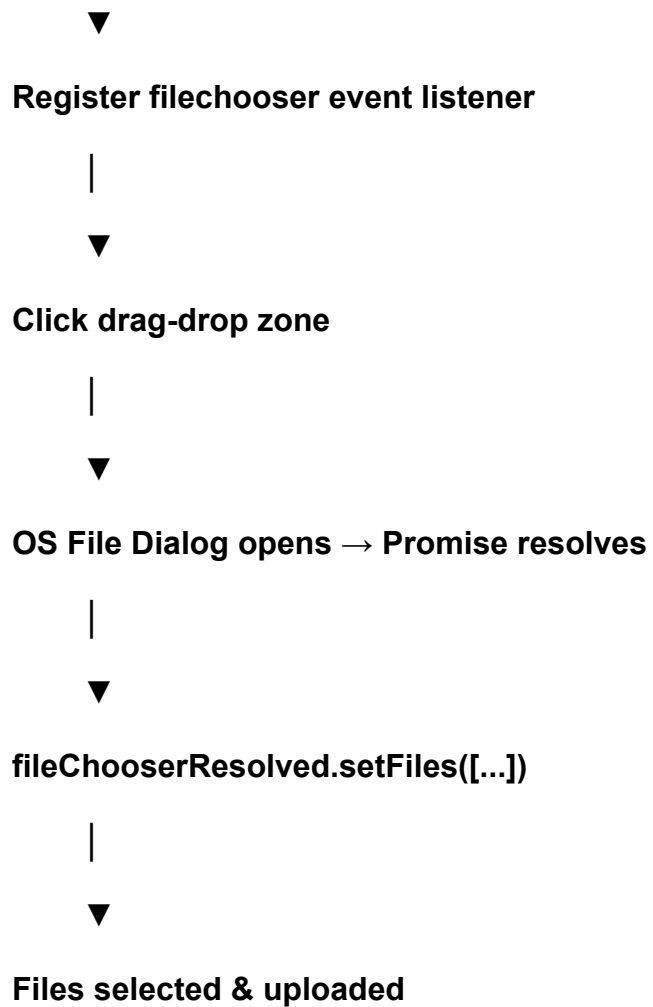
```
await fileChooserResolved.setFiles([  
  "to-upload/File1.txt",  
  "to-upload/File2.xlsx",  
  "to-upload\\File3.docx"  
])
```

- Uses the resolved FileChooser object to select multiple files
- Same array syntax as Test 1

Test 2 Flow

Navigate to page

|



Approach 1 vs Approach 2 Comparison

Aspect	Test 1 (<code>setInputFiles</code>)	Test 2 (FileChooser)
Works with	Standard <code><input type="file"></code>	Drag-drop / custom upload zones

Dialog opens?	No — directly sets files	Yes — OS file dialog
Code complexity	Simple, one line	Requires Promise setup
Clear files	<code>setInputFiles([])</code>	N/A
Best for	Traditional file inputs	Modern drag-and-drop UIs

`setInputFiles()` Options

// Single file

```
await input.setInputFiles("path/to/file.txt");
```

// Multiple files

```
await input.setInputFiles(["file1.txt", "file2.xlsx"]);
```

// Clear files

```
await input.setInputFiles([]);
```

```
// With file object (no actual file needed on disk)

await input.setInputFiles({
  name: "test.txt",
  mimeType: "text/plain",
  buffer: Buffer.from("file content here")
});
```

Key Takeaways

Concept	Detail
<code>setInputFiles()</code>	Direct way to upload via standard file input
<code>waitForEvent("filechooser")</code>	Handles OS file dialog triggered by click
Promise before click	Listener must be registered BEFORE the dialog opens
Empty array <code>[]</code>	Clears all selected files
Forward slash <code>/</code>	Use over <code>\\</code> for cross-platform path compatibility

When to Use `waitForEvent("filechooser")` vs `setInputFiles()`

The deciding factor is what type of upload element the page uses.

Use `setInputFiles()` When...

The page has a standard HTML file input element:

```
<!-- Standard file input — visible in DOM -->
```

```
<input type="file" id="filesToUpload" />
```

```
<input type="file" multiple />
```

```
<input type="file" accept=".pdf,.docx" />
```

```
// Direct and simple — no dialog involved
```

```
await page.locator("#filesToUpload").setInputFiles("file.txt");
```

How to identify it in DevTools:

Elements Tab → look for:

```
<input type="file"> ← setInputFiles() works here
```

Use `waitForEvent("filechooser")` When...

The page uses a custom upload trigger that opens the OS file dialog:

<!-- Custom upload zones — NO visible input in DOM -->

<div id="drag-drop-upload">Drop files here</div>

<button id="upload-btn">Choose File</button>

<div class="dropzone">Click to Upload</div>

// Must use filechooser event — dialog opens on click

const fileChooserPromise = page.waitForEvent("filechooser");

await page.locator("#drag-drop-upload").click();

const fileChooser = await fileChooserPromise;

await fileChooser.setFiles("file.txt");

How to identify it in DevTools:

Elements Tab → look for:

<div>, <button>, with upload behavior

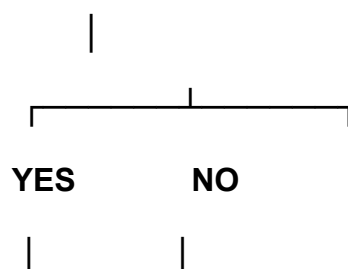
← No <input type="file"> visible

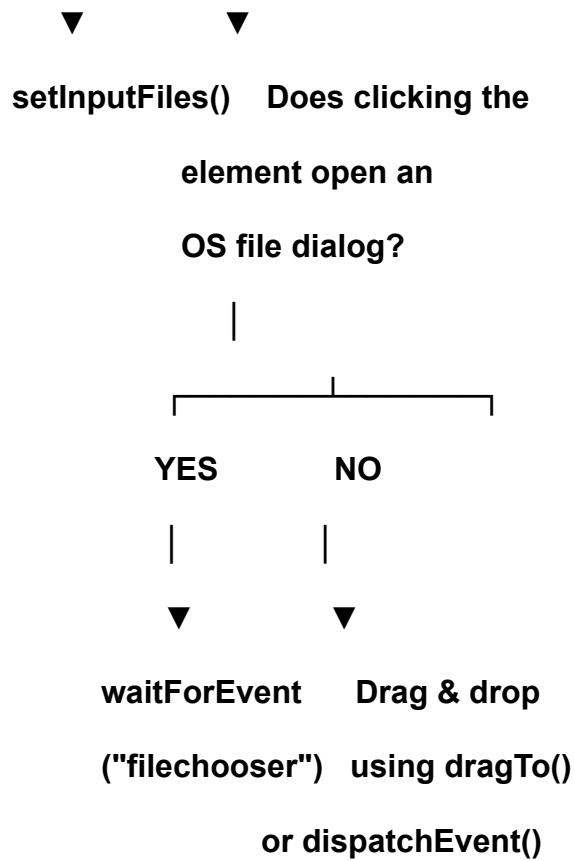
← waitForEvent("filechooser") works here

Decision Tree

Is there an <input type="file">

directly in the DOM?





Real World Examples

Website/Element Type	Method to Use
Standard <code><input type="file"></code>	<code>setInputFiles()</code>
Drag and drop zone (div/span)	<code>waitForEvent ("filechooser")</code>

Custom styled upload button	<code>waitForEvent t("filechoo ser")</code>
--------------------------------	---

Hidden file input triggered by button	<code>waitForEvent t("filechoo ser")</code>
--	---

React/Angular dropzone components	<code>waitForEvent t("filechoo ser")</code>
---	---

Quick Test to Identify Which to Use

// Run this in browser DevTools console

// on the upload element

```
document.querySelector('#your-upload-element').tagName
```

// Returns "INPUT" → use setInputFiles()

// Returns "DIV" → use waitForEvent("filechooser")

// Returns "BUTTON" → use waitForEvent("filechooser")

Common Mistake

// ❌ Wrong — trying setInputFiles() on a div

```
await page.locator("#drag-drop-upload").setInputFiles("file.txt");
```

// Error: Element is not an <input> — throws error

//  Right — use `waitForEvent` for non-input elements

```
const fileChooserPromise = page.waitForEvent("filechooser");
```

```
await page.locator("#drag-drop-upload").click();
```

```
const fileChooser = await fileChooserPromise;
```

```
await fileChooser.setFiles("file.txt");
```

Key Takeaways

	<code>setInputFiles()</code>	<code>waitForEvent("filechooser")</code>
Element type	<code><input type="file"></code>	<code><div></code> , <code><button></code> , custom elements
OS dialog opens?	No	Yes
Complexity	Simple	Needs Promise setup before click
How to	Visible in DOM as <code>input</code> tag	No <code>input</code> tag — custom UI component

ident
ify

Golden Rule — if clicking the upload area opens your operating system's file picker dialog, use `waitForEvent("filechooser")`. If it's a plain HTML file input, use `setInputFiles()`.

Tags in Playwright | Playwright Tutorials

"Tag" refers to a label or identifier that is attached to a particular test.

Usage -

- Sometimes you want to tag your tests and then filter by tag in the test report.
- Or you might want to only run tests that have a certain tag.

To tag a test, either add @tag-name to the test title, or provide an additional details object when declaring a test. Note that tags must start with @ symbol.

You can run tests that have a particular tag with --grep command line option `npx playwright test --grep=@tag` you can skip the tests with a certain tag: by using command `npx playwright test --grep-invert=@tag`

Note: In your `playwright.config.ts` file under `defineconfig` section, you can also specify `grep:/ui/` to execute all tests that contain the UI tag or `grepInvert:/ui/` to skip all the tests containing the UI tag

In VSCode:

Create a file under the tests folder called `TagsPractice.spec.ts` and include the following code:

```
import {test} from "@playwright/test"

test.describe("Describe Block 1",{tag:"@sanity"}, async()=>{

    test("Practice Test1 @UI",async({})=>{

        console.log("Practice Test 1")

    })
})
```

```

    test("Practice Test2 @API", async({}) => {

        console.log("Practice Test 2")

    })

    test("Practice Test3 @UI @smoke", async({page}) => {

        console.log("Practice Test 3")

    })
})

test("Practice
Test4", {tag:["@UI", "@smoke", "@DB"]}, async({}) => {

    console.log("Practice Test 4")

})

test("Practice Test5", {tag:"@API"}, async({}) => {

    console.log("Practice Test 5")

})

test("Practice Test6", {tag:"@API"}, async({}) => {

    console.log("Practice Test 6")

})

```

Explanation:

Code Explanation

This file demonstrates test organization using **describe** blocks and tags in Playwright.

Two Ways to Add Tags in Playwright

Method 1 — In test title	Method 2 — In options
test("My Test @UI", ...)	test("My Test",
	{tag: "@UI"}, ...)
Simple but informal	Official Playwright way

test.describe Block

```
test.describe("Describe Block 1", { tag: "@sanity" }, async () => {
```

- **test.describe** groups related tests together
 - **"Describe Block 1"** — name of the group
 - **{ tag: "@sanity" }** — applies **@sanity** tag to all tests inside this block
 - All 3 tests inside inherit the **@sanity** tag automatically
-

Test 1 — Title Tag Only

```
test("Practice Test1 @UI", async ({}) => {
```



```
    console.log("Practice Test 1")
  })
```

- Tag **@UI** is added directly in the title string
 - Also inherits **@sanity** from the describe block
 - Effective tags: **@sanity @UI**
 - **{}** — empty destructure, no fixtures needed
-

Test 2 — Title Tag Only

```
test("Practice Test2 @API", async ({}) => {
  console.log("Practice Test 2")
})
```

- Tag **@API** added in title string
 - Inherits **@sanity** from describe block
 - Effective tags: **@sanity @API**
-

Test 3 — Multiple Title Tags

```
test("Practice Test3 @UI @smoke", async ({ page }) => {
  console.log("Practice Test 3")
})
```

- Two tags in the title: **@UI** and **@smoke**
 - Inherits **@sanity** from describe block
 - **page** fixture is destructured (even though unused here)
 - Effective tags: **@sanity @UI @smoke**
-

Standalone Tests (Outside Describe)

Test 4 — Options Object with Array of Tags

```
test("Practice Test4", { tag: ["@UI", "@smoke", "@DB"] }, async ({})) => {  
  
    console.log("Practice Test 4")  
  
})
```

- Uses the official options object { tag: [...] }
 - Passes multiple tags as an array
 - No describe block — not part of any group
 - Effective tags: @UI @smoke @DB
-

Test 5 — Options Object with Single Tag

```
test("Practice Test5", { tag: "@API" }, async ({})) => {  
  
    console.log("Practice Test 5")  
  
})
```

- Single tag passed as a string in options object
 - Effective tags: @API
-

Test 6 — Same as Test 5

```
test("Practice Test6", { tag: "@API" }, async ({})) => {  
  
    console.log("Practice Test 6")  
  
})
```

- Effective tags: @API

Tag Summary Table

Test	Description Tag	Owner Tags	Effective Tags
Test 1	@sanity	@UI	@sanity @UI
Test 2	@sanity	@API	@sanity @API
Tests	@san	@UI @	@sanity @UI

t 3	i t y	s m o k e	@smo ke
--------	-------------	-----------------------	------------

1 e s t 4	n o n e	@ U I @ s m o k e @ D B	@UI @smo ke @DB
-----------------------	------------------	--	--------------------------

1 e s t 5	n o n e	@ A P I	@API
-----------------------	------------------	------------------	------

1 e s t 6	n o n e	@ A P I	@API
-----------------------	------------------	------------------	------

Running Tests by Tag

Run only @sanity tests (Test 1, 2, 3)

npx playwright test --grep @sanity

Run only @UI tests (Test 1, 3, 4)

npx playwright test --grep @UI

Run only @API tests (Test 2, 5, 6)

npx playwright test --grep @API

Run @smoke tests (Test 3, 4)

npx playwright test --grep @smoke

Run multiple tags (OR condition)

npx playwright test --grep "@UI|@API"

Exclude a tag

npx playwright test --grep-invert @API

Combine include and exclude

npx playwright test --grep @UI --grep-invert @smoke

Tag Naming Conventions

// Common tag categories used in real projects

@sanity // Quick smoke tests to verify basic functionality

@smoke // Core critical path tests

@UI // Frontend/browser tests

@API // Backend/API tests

@DB // Database tests

@regression // Full regression suite

@slow // Tests that take long to run

Key Takeaways

Concept	Detail
test.describe tag	Applies to all tests inside the block
Title tags (@tag)	Informal — added directly in test name string
Options tags ({tag: ...})	Official Playwright way — string or array
Tag inheritance	Tests inside describe inherit the describe tag
--grep	CLI flag to filter and run tests by tag
{}	Empty fixture destructure when no fixtures are needed

Playwright Tags — In Simple Terms

What is a Tag?

Think of tags like labels on folders in a filing cabinet.

Filing Cabinet = Your Test Suite

📁 Test 1 → 🏷️ Quick 🏷️ Login
📁 Test 2 → 🏷️ Slow 🏷️ Payment
📁 Test 3 → 🏷️ Quick 🏷️ Login 🏷️ Payment
📁 Test 4 → 🏷️ Slow 🏷️ Login

- You can pull out all "Quick" folders at once
 - Or all "Login" folders
 - Or all "Quick Login" folders together
-

Real Life Analogy

Imagine you have 100 tests in your project:

Without Tags

With Tags

Run ALL 100 tests
every single time

Run only the 10 @smoke tests
when you need a quick check

Takes 2 hours ❌

Takes 10 minutes ✅

Tags let you say "just run THESE tests" instead of everything

Why Do We Need Tags?

In real projects tests are grouped by purpose:

@smoke → "Is the app basically working?" (run every deployment)

@sanity → "Do the most critical things work?" (run after a fix)

@UI → "Does the screen look and work right?"

@API → "Is the backend responding correctly?"

@slow → "These take long — run only at night"

@DB → "Does the database work correctly?"

How It Works — Simple Example

```
test("Login works", { tag: "@smoke" }, async ({ page }) => { ... })
```

```
test("Payment works", { tag: "@slow" }, async ({ page }) => { ... })
```

```
test("Profile loads", { tag: "@smoke" }, async ({ page }) => { ... })
```

Run ONLY smoke tests → runs "Login works" + "Profile loads"

```
npx playwright test --grep @smoke
```

Run ONLY slow tests → runs "Payment works"

```
npx playwright test --grep @slow
```

Everyday Scenarios

Scenario 1 — Developer just made a small fix

Don't run all 100 tests — just run quick sanity checks

```
npx playwright test --grep @sanity
```

Done in 5 mins instead of 2 hours 

Scenario 2 — Before a big release

Run everything EXCEPT the slow tests

```
npx playwright test --grep-invert @slow
```

Scenario 3 — Only test the login feature

```
npx playwright test --grep @login
```

Scenario 4 — Run UI and API tests together

```
npx playwright test --grep "@UI|@API"
```

Tag Inheritance from Describe

```
test.describe("Login Tests", { tag: "@login" }, () => {  
  test("Valid login @smoke", ...) // has @login AND @smoke  
  test("Invalid login", ...)      // has @login only  
  test("Forgot password", ...)    // has @login only  
})
```

Like putting all folders into a drawer labeled "Login" — every folder in that drawer automatically gets the Login label

Summary

Question	Answer
What are tags?	Labels you put on tests
Why use them?	To run only specific tests when needed
How to add?	<code>{ tag: "@tagname" }</code> or write <code>@tagname</code> in test title
How to run by tag?	<code>npx playwright test --grep @tagname</code>
Can one test have many tags?	Yes — <code>{ tag: ["@UI", "@smoke", "@fast"] }</code>
Do describe tags apply to all tests inside?	Yes — automatically inherited

Bottom line — tags are just a way to say *"put this test in this group"* so you can run just that group whenever you want, without running everything.

Code Explanation

This file demonstrates test organization using `describe` blocks and tags in Playwright.

What Are Tags?

Tags are labels attached to tests or groups of tests that let you:

- Run only specific subsets of tests
- Organize tests by type, feature, or priority
- Filter tests from the command line

// Tags can be added two ways:

// 1. In the test title string

```
test("Practice Test1 @UI", ...)
```

// 2. In the test options object

```
test("Practice Test4", { tag: "@UI" }, ...)
```

test.describe Block

```
test.describe("Describe Block 1", { tag: "@sanity" }, async () => {
```

- Groups related tests under "Describe Block 1"
- `{ tag: "@sanity" }` applies the `@sanity` tag to all tests inside this block
- Acts like a parent tag — every child test inherits it

Test 1 — Title Tag Only

```
test("Practice Test1 @UI", async ({}) => {  
  console.log("Practice Test 1")  
})
```

- Tag is embedded directly in the test name string — **@UI**
 - Inherits **@sanity** from the describe block
 - Effective tags: **@sanity, @UI**
 - **async ({})** — empty destructure, no fixtures needed
-

Test 2 — Title Tag Only

```
test("Practice Test2 @API", async ({}) => {  
  console.log("Practice Test 2")  
})
```

- Tag in title: **@API**
 - Inherits **@sanity** from describe block
 - Effective tags: **@sanity, @API**
-

Test 3 — Multiple Title Tags

```
test("Practice Test3 @UI @smoke", async ({ page }) => {  
  console.log("Practice Test 3")  
})
```

- Two tags in title: **@UI** and **@smoke**
 - Inherits **@sanity** from describe block
 - Effective tags: **@sanity, @UI, @smoke**
 - Uses **page** fixture unlike Tests 1 & 2
-

Tag Inheritance Visualization

```
test.describe (@sanity)
```

```

|
|— Test1 (@UI)
|   └─ effective: @sanity, @UI
|
|
|— Test2 (@API)
|   └─ effective: @sanity, @API
|
|
└─ Test3 (@UI, @smoke)
    └─ effective: @sanity, @UI, @smoke

```

Standalone Tests

Test 4 — Options Object with Multiple Tags

```

test("Practice Test4", { tag: ["@UI", "@smoke", "@DB"] }, async ({})
=> {

  console.log("Practice Test 4")

})

```

- Uses options object { **tag: [...]** } with an array for multiple tags
 - Outside any describe block — no inherited tags
 - Effective tags: **@UI**, **@smoke**, **@DB**
-

Test 5 & 6 — Single Tag via Options Object

```

test("Practice Test5", { tag: "@API" }, async ({}) => {

  console.log("Practice Test 5")

```

```
})
```

```
test("Practice Test6", { tag: "@API" }, async ({}) => {  
    console.log("Practice Test 6")  
})
```

- Both use options object with a single string tag
 - Outside describe block — no inherited tags
 - Effective tags: **@API**
-

Two Ways to Add Tags — Comparison

Method	Syntax	Example
Title string	<code>"Test Name @tag"</code>	<code>test("Login @UI", ...)</code>
Options object	<code>{ tag: "@tag" }</code>	<code>test("Login", {tag:"@UI"}, ...)</code>
Multiple via title	<code>"Test @tag1 @tag2"</code>	<code>test("Login @UI @smoke", ...)</code>

Multiple via options

```
{  tag: ["@t1", "@t2"]}
```

```
test("Login", {tag:["@UI","@smoke"]}, ...)
```

All Tests & Their Tags Summary

Test Description	Own Tags	Effective Tags
Describe Tag		
Sanity	@UI	@sanity, @UI
Sanity	@API	@sanity, @API

Sanity	@sanity,	@UI,	@sanity,
Smoke	@smoke,	@smoke,	@UI,
DB	@DB	@DB	@smoke
UI	@UI	@UI	
API	@API	@API	

Running Tests by Tag (CLI)

Run only @sanity tagged tests

`npx playwright test --grep @sanity`

Run only @UI tagged tests

`npx playwright test --grep @UI`

Run @UI AND @smoke tagged tests

```
npx playwright test --grep "@UI|@smoke"
```

Exclude @API tagged tests

```
npx playwright test --grep-invert @API
```

Run tests with multiple specific tags

```
npx playwright test --grep "@UI" --grep "@smoke"
```

Key Takeaways

Concept

Detail

test.describe

Groups tests and can apply shared tags

Tag inheritance

All tests inside describe inherit its tags

Titles
tags

@tagname directly in test name string

Options
tags

{ tag: "@tagname" } or **{ tag: ["@t1", "@t2"] }**

--
grep

CLI flag to filter and run tests by tag

Purpose

Organize and selectively run test subsets

Best practice — use tags to separate test types like **@smoke** (quick checks), **@sanity** (basic functionality), **@UI** (frontend), **@API** (backend), **@DB** (database) so you can run the right tests at the right time.

Flexible Test Execution: Running Tests with Either or Both Tags in Playwright | Playwright Tutorials

PowerShell Terminal is a command-line interface provided by Microsoft.

Batch Terminal refers to the command prompt (cmd.exe) provided by Microsoft Windows.

Bash Terminal is the default terminal on most Unix-based operating systems, including Linux and macOS.

To run tests containing either tag (logical OR operator):

Powershell Command -

```
npx playwright test --grep --% "@tag1|@tag2|@tag3"
```

Batch Command -

```
npx playwright test --grep "@tag1|@tag2"
```

Bash Command -

```
npx playwright test --grep "@tag1|@tag2|@tag3"
```

Run tests containing both tags (logical AND operator) using regex lookaheads:

```
npx playwright test --grep "(?=.*@tag1)(?=.*@tag2)(?=.*@tag3)"
```

Note: the batch command will be executed in the cmd in VSCode terminal. The regex lookahead commands can be executed in all 3 terminals: powershell, cmd (batch), or bash terminals in VSCode

